

QUESTIONS 20 / 20 completed

Bank

BANK

Write a Java program to implement a basic banking system using inheritance. Create a parent class called Account with attributes such as account number, account holder name, and balance. Then, create child classes such as SavingsAccount and CheckingAccount that inherit from the Account class and have their own unique methods for with drawal and

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.*;
2
3 class Bank {
4     public static void main(String[] args) {
5         Scanner scanner = new Scanner(System.in);
6         List<Account> accounts = new ArrayList<>();
7
8         accounts.add(new SavingsAccount("123456789", "Jane Doe", 5000, 1.5));
9         accounts.add(new CheckingAccount("987654321", "John Smith", 1000, 1000));
10
11         System.out.println("The accounts in the system are:");
12         for (int i = 0; i < accounts.size(); i++) {
13             System.out.println((i + 1) + ". " + accounts.get(i).getDetails());
14         }
15         while (true) {
16             System.out.print("\nPlease enter the number of the account you wish to perform a transaction on: ");
17             if (!scanner.hasNextInt()) break;
18             int accountChoice = scanner.nextInt();
19
20             if (accountChoice < 1 || accountChoice > accounts.size()) {
21                 System.out.println("Invalid account choice. Please enter a valid account number between 1 and " + accounts.size());
22             }
23         }
24     }
25 }
```

OUTPUT

```
The accounts in the system are:
1. Savings Account (123456789) - Jane Doe (Balance: $5000.00, Interest Rate: 1.5%)
2. Checking Account (987654321) - John Smith (Balance: $1000.00, Overdraft Limit: $1000.00)
```

```
Please enter the number of the account you wish to perform a transaction on:
```

INPUT

```
5000
7000
```

QUESTIONS 20 / 20 completed

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that inherit from the Employee class and have their own unique methods for managing teams.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Manager manager = new Manager("Alex", 101, 80000);
4         SalesPerson salesperson = new SalesPerson("Maria", 201, 45000);
5
6         System.out.println("---- Manager Details ----");
7         manager.displayEmployeeDetails();
8         manager.manageTeam("Development");
9
10        System.out.println("\n---- SalesPerson Details ----");
11        salesperson.displayEmployeeDetails();
12        salesperson.makeSales(25000);
13    }
14 }
15
16 class Employee {
17     protected String name;
18     protected int id;
19     protected double salary;
20
21     public Employee(String name, int id, double salary) {
```

OUTPUT

```
---- Manager Details ----
Employee Name: Alex
Employee ID: 101
Employee Salary: 80000.0
Alex is managing the Development team.

---- SalesPerson Details ----
Employee Name: Maria
Employee ID: 201
Employee Salary: 45000.0
```

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Animal-1

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and have their own unique methods for eating and reproducing.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Animal mammal = new Mammal("Lion", "Savanna");
4         Animal reptile = new Reptile("Snake", "Desert");
5         Animal bird = new Bird("Eagle", "Mountains");
6
7         System.out.println("----- Mammal -----");
8         mammal.displayDetails();
9         mammal.eat();
10        mammal.reproduce();
11
12        System.out.println("\n----- Reptile -----");
13        reptile.displayDetails();
14        reptile.eat();
15        reptile.reproduce();
16
17        System.out.println("\n----- Bird -----");
18        bird.displayDetails();
19        bird.eat();
20        bird.reproduce();
21    }
```

OUTPUT

```
----- Mammal -----
Animal Name: Lion
Habitat: Savanna
Lion (Mammal) eats by chewing and may drink milk.
Lion (Mammal) gives birth to live young.

----- Reptile -----
Animal Name: Snake
Habitat: Desert
Snake (Reptile) eats insects or small animals.
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Shapes-1

SHAPES-1

Write a Java program to implement a basic shape hierarchy using inheritance. Create a parent class called Shape with attributes such as color and area. Then, create child classes such as Circle, Square, and Triangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 class Shape {
3     String color;
4     double area;
5     Shape(String color) {
6         this.color = color;
7     }
8     public double calculateArea() {
9         return 0.0;
10    }
11
12    public double calculatePerimeter() {
13        return 0.0;
14    }
15
16    public void displayInfo() {
17        System.out.println("Color: " + color);
18        System.out.println("Area: " + calculateArea());
19        System.out.println("Perimeter: " + calculatePerimeter());
20    }
21 }
```

OUTPUT

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Library

LIBRARY

Write a Java program to implement a basic library management system using inheritance. Create a parent class called Item with attributes such as title and author. Then, create child classes such as Book and DVD that inherit from the Item class and have their own unique attributes and methods for borrowing and returning items.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Library {  
2     public static void main(String[] args) {  
3         Book book1 = new Book("The Alchemist", "Paulo Coelho", 208);  
4         DVD dvd1 = new DVD("Inception", "Christopher Nolan", 148);  
5  
6         book1.displayInfo();  
7         book1.borrowItem();  
8         book1.returnItem();  
9  
10        System.out.println();  
11  
12        dvd1.displayInfo();  
13        dvd1.borrowItem();  
14        dvd1.returnItem();  
15    }  
16 }  
17  
18 class Item {  
19     String title;  
20     String author;  
21     boolean isBorrowed;
```

OUTPUT

```
Title: The Alchemist, Author: Paulo Coelho  
Type: Book, Pages: 208  
The Alchemist has been borrowed.  
The Alchemist has been returned.  
  
Title: Inception, Author: Christopher Nolan  
Type: DVD, Duration: 148 mins  
Inception has been borrowed.  
Inception has been returned.
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

University

UNIVERSITY

Write a Java program to implement a basic university management system using inheritance. Create a parent class called Person with attributes such as name and age. Then, create child classes such as Student and Professor that inherit from the Person class and have their own unique attributes and methods for managing coursework and teaching classes.

PROGRAM EDITOR

Java

Run

Save

```
62 }
63
64 void manageCoursework() {
65     System.out.println("Student ID: " + studentId);
66     System.out.println("Enrolled Course: " + course);
67 }
68 }
69
70 class Professor extends Person {
71     String department;
72
73     Professor(String name, int age, String department) {
74         super(name, age);
75         this.department = department;
76     }
77
78     void teachClass() {
79         System.out.println("Department: " + department);
80         System.out.println("Status: Currently teaching classes in " + department);
81     }
82 }
```

OUTPUT

```
Exception in thread "main" java.lang.NumberFormatException: For input string:
"Age:20"
```

INPUT

Name:Ganesh
Age:20
Student ID:s101
Course:computer
Science

QUESTIONS 20 / 20 completed

Games

GAMES

Write a Java program to implement a basic game system using inheritance. Create a parent class called Game with attributes such as title and genre. Then, create child classes such as ActionGame and PuzzleGame that inherit from the Game class and have their own unique attributes and methods for playing the game.

PROGRAM EDITOR

Java

✓ Updated

Run

Save

```

48 }
49
50 class PuzzleGame extends Game {
51     int numberOfLevels;
52
53     PuzzleGame(String title, String genre, int numberOfLevels) {
54         super(title, genre);
55         this.numberOfLevels = numberOfLevels;
56     }
57
58     void solvePuzzle() {
59         System.out.println("Solving puzzles in " + title + "... Use your brain!");
60     }
61
62     void showDetails() {
63         displayInfo();
64         System.out.println("Number of Levels: " + numberOfLevels);
65     }
66 }

```

OUTPUT

```

--- Action Game Details ---
Title: Doom Eternal
Genre: First-Person Shooter
Difficulty Level: 5
Playing Doom Eternal! Get ready for some action!

--- Puzzle Game Details ---
Title: Tetris
Genre: Logic Puzzle

```

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Resturant

RESTURANT

Write a Java program to implement a basic restaurant management system using inheritance. Create a parent class called Menu with attributes such as name and price. Then, create child classes such as Appetizer and Entree that inherit from the Menu class and have their own unique attributes and methods for ordering and serving food.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.*;
2
3 class MainClass {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         System.out.println("=== Restaurant Management System ===");
8         System.out.print("Enter Appetizer name: ");
9         String appName = sc.nextLine();
10        System.out.print("Enter Appetizer price: ");
11        double appPrice = sc.nextDouble();
12        sc.nextLine();
13        System.out.print("Enter Appetizer special (unique attribute): ");
14        String appSpecial = sc.nextLine();
15
16        System.out.print("Enter Entree name: ");
17        String entName = sc.nextLine();
18        System.out.print("Enter Entree price: ");
19        double entPrice = sc.nextDouble();
20        sc.nextLine();
21    }
22 }
```

OUTPUT

```
=== Restaurant Management System ===
Enter Appetizer name: Enter Appetizer price: Enter Appetizer special (unique
attribute): Enter Entree name: Enter Entree price: Enter Entree spice level
(unique attribute):
--- Ordering Food ---
Ordered Appetizer: spring roll | Special: extra spicy
Ordered Entree: biriyani | Spice Level: medium

--- Serving Food ---
Serving Appetizer: spring roll | Enjoy the extra spicy !
```

INPUT

```
spring roll
105
extra spicy
biriyani
200
medium
```

QUESTIONS 20 / 20 completed

Forecasting

FORECASTING

Write a Java program to implement a basic weather forecasting system using inheritance. Create a parent class called Forecast with attributes such as date and temperature. Then, create child classes such as SunnyForecast and RainyForecast that inherit from the Forecast class and have their own unique attributes and methods for predicting the weather.

PROGRAM EDITOR

Java

Run

Save

```
36 }
37
38 void predict() {
39     System.out.println("Prediction: It's going to be a beautiful bright day!");
40     System.out.println("Sun Intensity Level: " + sunIntensity + "/10");
41 }
42 }
43
44 class RainyForecast extends Forecast {
45     double humidity;
46
47     RainyForecast(String date, double temperature, double humidity) {
48         super(date, temperature);
49         this.humidity = humidity;
50     }
51
52     void predict() {
53         System.out.println("Prediction: Grab an umbrella! Rain is expected.");
54         System.out.println("Humidity Level: " + humidity + "%");
55     }
56 }
```

INPUT

2023-10-25
32.5
8

OUTPUT

QUESTIONS 20 / 20 completed

Rental

RENTAL

Write a Java program to implement a basic car rental system using inheritance. Create a parent class called Vehicle with attributes such as make and model. Then, create child classes such as Sedan and SUV that inherit from the Vehicle class and have their own unique attributes and methods for renting and returning cars.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5
6         Sedan sedan = new Sedan("Toyota", "Camry", "TN-1234", 5);
7         SUV suv = new SUV("Ford", "Endeavour", "KA-5678", true);
8
9         System.out.println("=== Sedan Details ===");
10        sedan.displayDetails();
11        sedan.displaySedanInfo();
12        System.out.println("Price per day: $" + sedan.rentalPricePerDay());
13
14        System.out.println("\n=== SUV Details ===");
15        suv.displayDetails();
16        suv.displaySUVInfo();
17        System.out.println("Price per day: $" + suv.rentalPricePerDay());
18
19        System.out.println("\n--- Rental Operations ---");
20        // ... (more code) ...
21    }
```

OUTPUT

```
=== Sedan Details ===
Make: Toyota
Model: Camry
Plate Number: TN-1234
Status: Available
Passenger Capacity: 5
Price per day: $45.0

=== SUV Details ===
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Shape-2

SHAPE-2

Write a Java program to implement a basic shape hierarchy using polymorphism. Create a parent class called Shape with attributes such as area and perimeter. Then, create child classes such as Circle and Rectangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

PROGRAM EDITOR

Java

Run

Save

```
60
61     public void calculatePerimeter() {
62         perimeter = 2 * Math.PI * radius;
63     }
64 }
65 class Rectangle extends Shape {
66     double length;
67     double width;
68
69     public Rectangle(double length, double width) {
70         this.length = length;
71         this.width = width;
72     }
73
74     public void calculateArea() {
75         area = length * width;
76     }
77     public void calculatePerimeter() {
78         perimeter = 2 * (length + width);
79     }
80 }
```

OUTPUT

```
Enter choice: 1 for Circle, 2 for Rectangle
Enter radius:
--- Result ---
Area: 78.54
Perimeter: 31.42
```

INPUT

1
5

QUESTIONS 20 / 20 completed

Draw

DRAW

Write a Java program to demonstrate polymorphism with interfaces. Create an interface called Drawable with a method called draw(). Then, create classes such as Circle and Square that implement the Drawable interface and have their own unique implementations of the draw() method.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {  
2     public static void main(String[] args) {  
3  
4         Drawable d1 = new Circle();  
5         Drawable d2 = new Square();  
6  
7         d1.draw();  
8         d2.draw();  
9     }  
10 }  
11  
12 interface Drawable {  
13     void draw();  
14 }  
15 class Circle implements Drawable {  
16  
17     public void draw() {  
18         System.out.println("Drawing a Circle!");  
19     }  
20 }  
21 class Square implements Drawable {
```

OUTPUT

```
Drawing a Circle!  
Drawing a Square!
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Animal-2

ANIMAL-2

Write a Java program to demonstrate polymorphism with abstract classes. Create an abstract class called Animal with abstract methods such as eat() and sleep(). Then, create child classes such as Dog and Cat that inherit from the Animal class and implement their own versions of the eat() and sleep() methods.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3
4         Animal a1 = new Dog();
5         Animal a2 = new Cat();
6
7         a1.displayType();
8         a1.eat();
9         a1.sleep();
10
11        System.out.println();
12
13        a2.displayType();
14        a2.eat();
15        a2.sleep();
16    }
17 }
18
19 abstract class Animal {
20     abstract void eat();
21     abstract void sleep();
22 }
```

OUTPUT

System Error: Could not reach compiler or server error occurred.

INPUT

This is animal
dog eats bones
dog sleeps in a
kennel

QUESTIONS 20 / 20 completed

Overloading

OVERLOADING

Write a Java program to demonstrate method overloading with variable arguments. Create a method called `sum()` that takes in a variable number of integers and returns their sum. Then, create an overloaded version of the `sum()` method that takes in a variable number of doubles and returns their sum.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     static int sum(int... nums) {
3         int total = 0;
4         for (int n : nums) {
5             total += n;
6         }
7         return total;
8     }
9     static double sum(double... nums) {
10        double total = 0.0;
11        for (double n : nums) {
12            total += n;
13        }
14        return total;
15    }
16    public static void main(String[] args) {
17        System.out.println("Sum of ints: " + sum(1, 2, 3, 4, 5));
18        System.out.println("Sum of doubles: " + sum(1.5, 2.5, 3.0));
19    }
20 }
```

OUTPUT

```
Sum of ints: 15
Sum of doubles: 7.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Overriding

OVERRIDING

Write a Java program to demonstrate method overriding with a simple calculator. Create a parent class called Calculator with methods such as add(), subtract(), multiply(), and divide(). Then, create a child class called ScientificCalculator that overrides the multiply() method to perform a more complex calculation.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Calculator basicCalc = new Calculator();
4         System.out.println("--- Basic Calculator ---");
5         basicCalc.add(10, 5);
6         basicCalc.subtract(10, 5);
7         basicCalc.multiply(10, 5);
8         basicCalc.divide(10, 5);
9
10        System.out.println();
11
12
13        ScientificCalculator sciCalc = new ScientificCalculator();
14        System.out.println("--- Scientific Calculator ---");
15        sciCalc.add(10, 5);
16        sciCalc.subtract(10, 5);
17        sciCalc.multiply(10, 5);
18        sciCalc.divide(10, 5);
19    }
20 }
21
```

OUTPUT

```
--- Basic Calculator ---
Addition Result: 15.0
Subtraction Result: 5.0
Multiplication Result: 50.0
Division Result: 2.0

--- Scientific Calculator ---
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Person

PERSON

Write a Java program to create a class called Person with a constructor that takes in a name and age, and a method to print out the person's name and age.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Person person = new Person("Ganesh", 20);
4         person.printDetails();
5     }
6 }
7 class Person {
8     private String name;
9     private int age;
10
11     public Person(String name, int age) {
12         this.name = name;
13         this.age = age;
14     }
15     public void printDetails() {
16         System.out.println("Name: " + name + ", Age: " + age);
17     }
18 }
```

OUTPUT

Name: Ganesh, Age: 20

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Rect-Const

RECT-CONST

Write a Java program to create a class called Rectangle with a constructor that takes in the length and width of the rectangle, and a method to calculate and return the area of the rectangle.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5
6         if (sc.hasNextDouble()) {
7             double l = sc.nextDouble();
8             if (sc.hasNextDouble()) {
9                 double w = sc.nextDouble();
10
11                 Rectangle rect = new Rectangle(l, w);
12
13                 System.out.println(rect.calculateArea());
14             }
15         }
16         sc.close();
17     }
18 }
19
20 class Rectangle {
21     double length;
```

OUTPUT

50.0

INPUT

5 10

QUESTIONS 20 / 20 completed

Bank-Const

BANK-CONST

Write a Java program to create a class called BankAccount with a constructor that takes in an account number and an initial balance, and methods to deposit and withdraw money from the account.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {  
2     public static void main(String[] args) {  
3  
4         BankAccount account = new BankAccount("1234567890", 1000.0);  
5  
6         account.deposit(500);  
7         account.withdraw(300);  
8  
9         System.out.println("Account Number: " + account.getAccountNumber());  
10        System.out.println("Final Balance: " + account.getBalance());  
11    }  
12 }  
13  
14 class BankAccount {  
15     private String accountNumber;  
16     private double balance;  
17  
18     public BankAccount(String accountNumber, double initialBalance) {  
19         this.accountNumber = accountNumber;  
20         this.balance = initialBalance;  
21     }  
22 }
```

OUTPUT

```
Successfully deposited: 500.0  
Successfully withdrawn: 300.0  
Account Number: 1234567890  
Final Balance: 1200.0
```

INPUT

Your INPUT goes
here!

QUESTIONS 20 / 20 completed

Car-Const

CAR-CONST

Write a Java program to create a class called Car with a constructor that takes in the make, model, and year of the car, and a method to print out the car's make, model, and year.

PROGRAM EDITOR

Java

Run

Save

```
1 public class Main {
2     public static void main(String[] args) {
3         Car car1 = new Car("Toyota", "Corolla", 2022);
4         car1.printDetails();
5     }
6 }
7
8 class Car {
9     private String make;
10    private String model;
11    private int year;
12    public Car(String make, String model, int year) {
13        this.make = make;
14        this.model = model;
15        this.year = year;
16    }
17
18    public void printDetails() {
19        System.out.println("Make: " + make);
20        System.out.println("Model: " + model);
21        System.out.println("Year: " + year);
22    }
23 }
```

OUTPUT

```
Make: Toyota
Model: Corolla
Year: 2022
```

INPUT

Your INPUT goes here!

QUESTIONS 20 / 20 completed

Student-Const

STUDENT-CONST

Write a Java program to create a class called Student with a constructor that takes in a name, ID number, and a list of grades, and methods to calculate and return the student's average grade and letter grade.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.List;
2 public class Main {
3     public static void main(String[] args) {
4
5         List<Double> grades = List.of(85.0, 90.0, 78.0, 92.0, 88.0);
6
7         Student student = new Student(" Ganeshram", 192321060, grades);
8
9         System.out.println("Student Name: " + student.getName());
10        System.out.println("Student ID: " + student.getIdNumber());
11        System.out.printf("Average Grade: %.2f\n", student.calculateAverage());
12        System.out.println("Letter Grade: " + student.calculateLetterGrade());
13    }
14 }
15 class Student {
16     private String name;
17     private int idNumber;
18     private List<Double> grades;
19
20     public Student(String name, int idNumber, List<Double> grades) {
21         this.name = name;
```

OUTPUT

```
Student Name: Ganeshram
Student ID: 192321060
Average Grade: 86.60
Letter Grade: B
```

INPUT

Your INPUT goes
here!